



# SIMATS

---

## ENGINEERING



# SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

## Dynamic Playlists

Presented by  
S. Jaxon Rizal(192511421)



## Core Operations

A playlist must support `append(song)` frequently, `insert` at a position occasionally, `removeById(id)` when a user deletes, `move(id, newPos)` for reordering, and `getRange(start, length)` to feed the UI window. These operations vary in cost and access patterns: frequent appends and reads favor append-heavy structures, while frequent moves and mid-list inserts push toward balanced trees or skip lists. The





# Fast Access, Costly Reorders

Provides  $O(1)$  random access by index (`getByIndex`). Inserts/removals at an arbitrary position cost  $O(n)$  due to shifting. Removing by id needs a full  $O(n)$  search unless you maintain an id→index map, which is hard to keep consistent as items shift.

Best for append-heavy, read-dominant workloads. Frequent moves or arbitrary reorders incur high costs; index-based mutations become a bottleneck as shifts ripple through the list.

Baseline array-backed lists excel at reads, but are costly for mid-list moves.



## Mutations Fast

Baseline uses a doubly linked list with a hash map  $\text{id} \rightarrow \text{node}$ . Remove by id is  $O(1)$  via the map; unlinking is  $O(1)$ . Move by id is  $O(1)$  by unlinking and relinking. However, get by index requires walking from head, so  $O(n)$ . This is excellent for id-driven mutations (deletes, moves) but weak for UI scrolling or index-based access.





# Indexed Order-Stat

01

## Node per Item

Represent each playlist entry as a node with a stable unique id and payload (metadata, URI).

02

## Augment Node Sizes

For trees: store subtree sizes;  
for skip lists: store index spans  
to enable `select(k)` in  $O(\log n)$ .

03

## Mutations & Queries

Implement `insert-at-pos`,  
`remove-by-id`,  
`move(id,newPos)`, and  
`getRange(start,len)` with  
logarithmic costs.

# Index Costs

Get by index

Indexed tree / indexed skip list;  $n=100k$ ;  
 $O(\log n) \sim 17$

**$\approx 17$  steps**

Insert/Move by index

Indexed structures;  $n=100k$ ;  $O(\log n)$  per  
op

**$\approx 17$  steps**

Get by index (linked list)

Doubly linked list + hashmap;  $n=100k$ ;  
 $O(n)$  scans to mid

**$\approx 50k$  steps**

Mutations by id

Doubly linked list + hashmap;  
insert/remove/move by id

**$O(1)$**



## Indexing for Mix

Choose the indexed structure that handles mixed workloads: use stable IDs as the network sync primary key (UUIDs or DB IDs) to keep convergence clean. For single-client local performance, employ a size-augmented treap or an indexed skip list and validate with 10k–100k playlists. For multi-client or offline edits, consider sequence CRDTs (RGA/Logoot) or server-side order-statistic structures plus '



## Balanced Playlists

A suitable playlist structure must balance flexible mutations with reliable positional access. For mixed workloads, indexed structures like size-augmented trees or indexed skip lists deliver  $O(\log n)$  for add/remove/move and access by index. Choose stable IDs, maintain size metadata, and align concurrency with your multi-client model to keep performance predictable as playlists grow.





The background of the image is a light beige, textured surface. It is decorated with various musical symbols and instruments in a light gray color. These include a large treble clef on the left, several eighth and sixteenth notes scattered throughout, and a few musical staves with notes. The word "Thanks" is centered in a dark brown, serif font.

**Thanks**